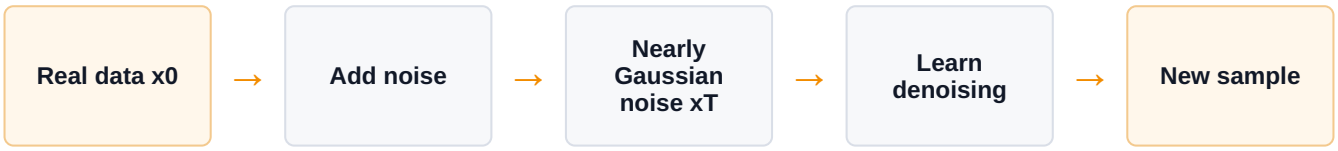
GENERATIVE AI - DIFFUSION MODELS

Diffusion Models: From Noise to Data

Intuition, forward and reverse diffusion, core mathematics, conditioning, architectures, use cases, and a tiny PyTorch model that trains in seconds.



Core idea. A diffusion model learns to reverse a known corruption process. During training we deliberately destroy structure with Gaussian noise. The neural network learns the statistical direction that removes that noise. At generation time we start from random noise and repeatedly denoise.

Question	Short answer
What is learned?	A neural network, usually parameterized by θ, that predicts noise, clean data, velocity, or a related denoising quantity.
What is fixed?	The forward noise schedule is usually chosen in advance rather than learned.
Why many steps?	Each reverse step is easier than generating a complicated sample in one jump.
Where is it used?	Image generation/editing, audio, video, super-resolution, scientific generation, and more.

This chapter uses DDPM-style diffusion as the main teaching model, then shows how the idea evolves into latent diffusion and modern conditional systems.

1. WHY DIFFUSION?

Why generate by destroying first?

Diffusion turns a hard one-step generation problem into many easier local denoising problems.

The generation problem

Suppose the data distribution contains faces, landscapes, handwriting, speech, or molecules. Sampling directly from that complex high-dimensional distribution is difficult. Diffusion introduces a simple path between **data** and **Gaussian noise**.

Hard view

Map one random vector directly to a perfect image in one large nonlinear jump.

Diffusion view

Learn a sequence of small corrections: slightly noisy → slightly cleaner → ... → data.

Statistical thought process

1. Choose a corruption process whose probabilities are easy to compute.
2. Train the model on many noise levels so it learns what structure looks like at each level.
3. At sampling time, approximately invert the corruption process.

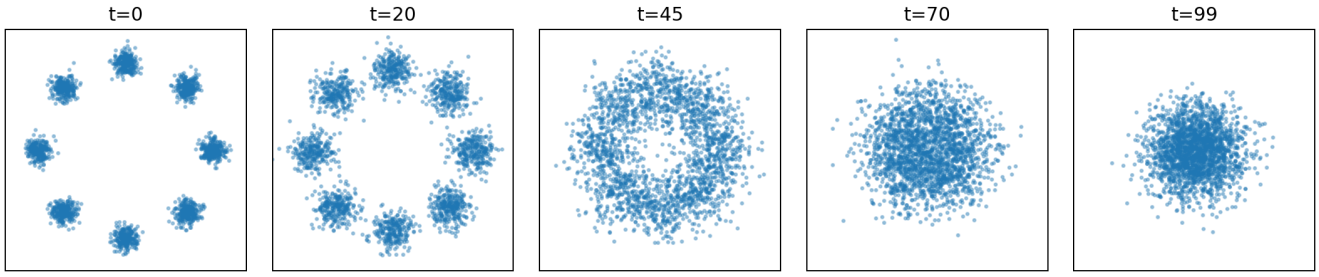
Analogy. It is easier to teach someone how to restore a slightly damaged drawing repeatedly than to ask them to draw an unknown masterpiece from a blank page in one step.

2. FORWARD PROCESS

Forward diffusion: gradually add noise

The forward process q is not the generative model. It is the controlled corruption process that creates training examples.

Starting with clean data x_0, we add a small amount of Gaussian noise at each timestep. As t increases, information about the original sample is progressively lost.



Toy 2-D data as noise increases. The final distribution approaches a simple Gaussian-like cloud.

x0: clean → **x20** → **x45** → **x70** → **xT: noise**

Important. We never need to run every forward step during training. A closed-form equation lets us jump from x0 directly to any chosen xt.

3. BASIC MATH

One forward diffusion step

Each step keeps most of the previous signal and injects a small amount of fresh Gaussian noise.

$$q(x_t \mid x_{t-1}) = N(\sqrt{\alpha_t}\, x_{t-1}, \beta_t I)$$

We define β_t as the noise variance at step t and $\alpha_t = 1 - \beta_t$. A sample can be written more directly as:

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon, \quad \varepsilon \sim N(0, I)$$

Symbol	Meaning	Intuition
β_t	Noise added at step t	Small β means gentle corruption.
α_t	$1 - \beta_t$	How much signal is retained.
ε	Standard Gaussian noise	A random direction of corruption.
I	Identity covariance	Independent unit-scale noise per dimension before scaling.

Why the square roots? Variances add through squared scale factors. Scaling noise by √β gives it variance β.

4. CLOSED-FORM NOISING

Jump directly from x0 to xt

The cumulative signal retention α-bar lets us create a training pair at any noise level in one operation.

Define the cumulative product:

$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$$

Then the result of many small Gaussian corruption steps has a simple closed form:

$$q(x_t \mid x_0) = N(\sqrt{\bar{\alpha}_t} x_0, (1-\bar{\alpha}_t) I)$$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{(1-\bar{\alpha}_t)} \varepsilon$$

Example

If $\bar{\alpha}_t = 0.64$, then √0.64 = 0.8 and √0.36 = 0.6. Therefore:

$$x_t = 0.8 x_0 + 0.6 \varepsilon$$

The sample still contains substantial signal, but random noise is now mixed into every dimension.

Teaching shortcut. Students only need to remember that α-bar tells us the cumulative amount of signal that survives until timestep t.

5. SCHEDULES

What does the noise schedule control?

The schedule decides how quickly information is destroyed across timesteps.

A **noise schedule** is the sequence $\beta_1, ..., \beta_T$. Early DDPMs commonly used simple linear schedules; later systems often use cosine-like or other schedules that allocate signal-to-noise levels more usefully.

Choice	Effect	Reasoning
Small T	Fewer denoising steps	Faster, but each reverse step must make a larger correction.
Large T	Many gentle steps	Easier local transitions but more sampling cost.
Noise too fast	Signal disappears early	Middle timesteps become less informative.
Noise too slow	Late state may retain structure	Starting distribution is less like simple Gaussian noise.

Signal-to-noise viewpoint

$$\mathbf{SNR(t) \approx \bar{\alpha}_t / (1 - \bar{\alpha}_t)}$$

Early timesteps have high signal-to-noise ratio; late timesteps have low signal-to-noise ratio. The same neural network must learn to denoise across this spectrum.

6. REVERSE PROCESS

The learned reverse diffusion process

Generation requires moving from noise back toward the data distribution.

If the exact reverse conditional were known, we could sample x_{t-1} from x_t. In real data this reverse distribution is unknown, so a neural network approximates it.

$$p_\theta(x_{t-1} \mid x_t) = N(\mu_\theta(x_t, t), \Sigma_t)$$



What does θ mean?

θ represents all learnable neural-network parameters: convolution kernels, linear-layer weights, attention weights, normalization parameters, and so on. **These are what gradient descent changes during training.**

No separate model per timestep. Usually one network is shared across all t. The timestep is supplied as an input embedding so the same weights behave differently at different noise levels.

7. PARAMETERIZATION

Why predict the noise?

Instead of directly predicting a complete clean sample, DDPM training gives the model a simple supervised target: the Gaussian noise we added.

During training we sampled the noise ourselves, so we know the exact target ε. The model receives x_t and t and predicts:

$$\varepsilon_\theta(x_t, t) \approx \varepsilon$$

Once the predicted noise is known, the reverse-process mean can be calculated. This turns generative modeling into a familiar regression task.

Noise prediction (ε)

Classic DDPM teaching parameterization. Target is exactly the noise used to corrupt x0.

Other parameterizations

Some systems predict x0 directly, a velocity-like v target, or a score. They encode closely related information but can differ in optimization behavior.

Score interpretation

The denoiser is closely related to the **score**, the gradient of log probability with respect to the noisy sample. Roughly:

$$\nabla_x \log q_t(x) \approx -\varepsilon_\theta(x,t) / \sqrt{(1-\bar{\alpha}_t)}$$

The score points toward regions of higher probability density, which gives another intuition for why repeated denoising moves random samples toward the data manifold.

8. TRAINING OBJECTIVE

The basic loss is surprisingly simple

Choose a data point, choose a noise level, add known noise, and teach the network to recover that noise.

$$L = E_{x0,t,\varepsilon}[\ ||\ \varepsilon - \varepsilon_\theta(x_t,t)\ ||^2\ ]$$

One training example

1. Sample a clean data item x0.
2. Sample timestep t uniformly or from a chosen weighting.
3. Sample Gaussian noise ε.
4. Create xt using the closed-form noising equation.
5. Predict ε_θ(xt,t).
6. Compute mean squared error against the true ε.
7. Backpropagate and update θ.

```
loss = ((predicted_noise - true_noise) ** 2).mean()
optimizer.zero_grad()
loss.backward()          # compute dLoss / dTheta
optimizer.step()         # change the neural-network weights
```

What changes during training? The forward schedule stays fixed. The random noise changes every batch. The neural-network weights change so that its noise estimate becomes statistically better across the whole dataset and across all timesteps.

9. WHAT THE MODEL LEARNS

Different timesteps teach different skills

The timestep input tells one shared denoiser how much structure is expected to remain.

Noise level	What xt looks like	What the network must learn
Low t	Almost clean	Fine details, textures, tiny perturbations.
Middle t	Structure + substantial noise	Shapes, parts, local/global consistency.
High t	Mostly noise	Coarse global structure and statistical direction toward likely data.

Why timestep conditioning matters

The same noisy vector can mean different things depending on the amount of corruption. A timestep embedding gives the network a representation of t that can modulate intermediate features.



Connection to transformers. Sinusoidal timestep embeddings use the same broad idea as sinusoidal positional features: map a scalar index into many frequencies so the network can distinguish nearby and far-apart values smoothly.

10. SAMPLING

Generation: start from noise and iterate backward

Training sees random individual timesteps. Sampling chains the learned reverse transition from T to 0.

```
x = randn(sample_shape)              # x_T
for t in reversed(range(T)):
    eps = model(x, t)                # predict noise
    mean = reverse_mean(x, eps, t)   # estimate x_{t-1}
    if t > 0:
        x = mean + sigma_t * randn_like(x)
    else:
        x = mean
return x                             # generated x_0
```

Why generation is slower than a one-pass model

The neural network may be evaluated tens or hundreds of times per sample. Accelerated samplers reduce the number of evaluations, and latent-space models reduce the size of each evaluation.

Training	Sampling
Random t per example; highly parallelizable.	Timesteps are sequential because xt-1 depends on xt.
Known target noise.	No target; model generates its own trajectory.
Weights updated.	Weights frozen.

11. IMAGE ARCHITECTURE

From a toy MLP to an image U-Net

The diffusion equations stay almost the same; the denoising network changes to match the structure of the data.

2-D toy data

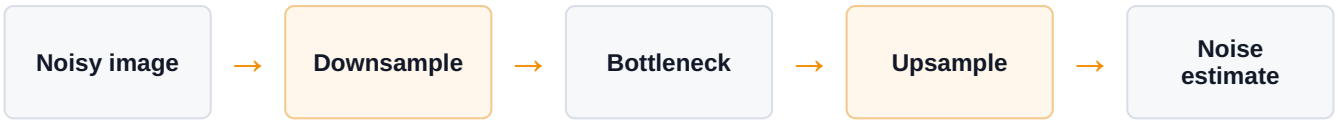
Input has only two numbers. A small multilayer perceptron is sufficient.

MLP

Images

Pixels have spatial structure at multiple scales. Convolutional U-Nets became a natural choice.

U-Net + attention



Why U-Net skip connections?

Downsampling captures large-scale structure but can lose exact spatial detail. Skip connections carry high-resolution features from the contracting path to matching layers in the expanding path. The model can therefore combine **global context** with **local detail**.

Modern alternative

Diffusion Transformer (DiT)-style systems replace much of the U-Net with transformer blocks operating on latent/image patches. The denoising objective remains diffusion-based; the backbone architecture changes.

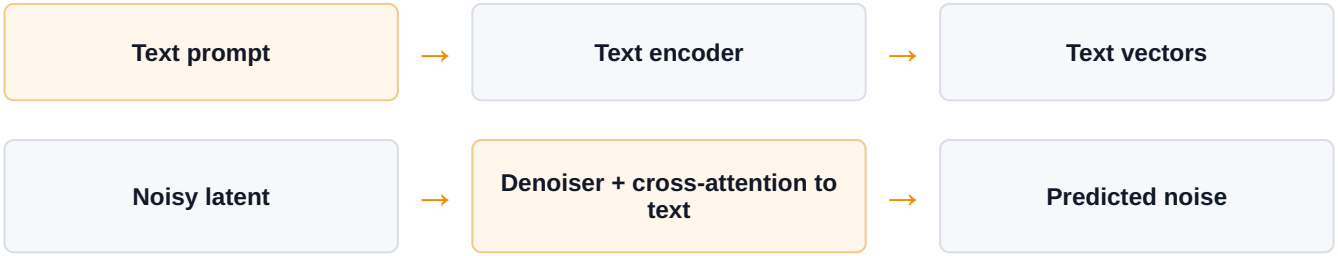
12. CONDITIONING

How text-to-image diffusion changes the model

Unconditional diffusion learns p(x). Conditional diffusion learns p(x | c), where c can be text, class labels, images, masks, or other signals.

$$\varepsilon_\theta(x_t, t) \rightarrow \varepsilon_\theta(x_t, t, c)$$

For text-to-image generation, the prompt is encoded into text embeddings. The image denoiser can use **cross-attention** so image/latent features query the text representation.



Interpretation

Without conditioning, the model asks: *What denoising direction makes this look like any likely training sample?* With text conditioning, it asks: *What denoising direction makes this both likely and compatible with the prompt?*

13. GUIDANCE

Classifier-free guidance

A practical way to strengthen conditioning at sampling time without training a separate classifier.

During training, the condition is sometimes intentionally removed. The same network therefore learns both conditional and unconditional denoising. At generation time we combine the two predictions:

$$\varepsilon_{guided} = \varepsilon_{uncond} + w(\varepsilon_{cond} - \varepsilon_{uncond})$$

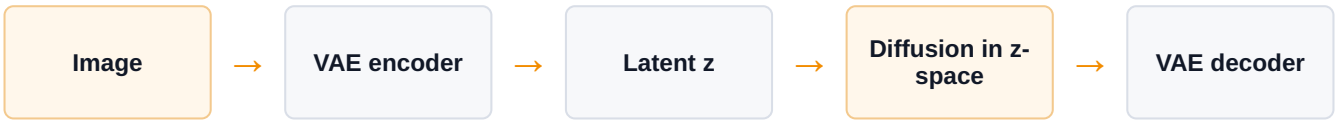
Guidance w	Typical effect
Near 0	Weak prompt influence; more like unconditional generation.
A moderate value	Stronger alignment with the condition.
Too high	Can reduce diversity or introduce artifacts/saturation depending on the model.

Vector intuition. The difference ($\varepsilon_{cond} - \varepsilon_{uncond}$) estimates a direction associated with the condition. Guidance scales that direction before the reverse step.

14. LATENT DIFFUSION

Why diffuse in latent space?

Pixel-space diffusion is expensive. Latent diffusion first compresses images, performs diffusion in the smaller representation, then decodes.



Why this matters

- A 512x512 RGB image has hundreds of thousands of scalar values.
- A learned latent representation can be spatially much smaller.
- The denoiser therefore performs fewer operations and uses less memory.
- The VAE handles compression/decompression; the diffusion model handles generation in the latent distribution.

Conceptual separation. The VAE is not the diffusion model. It defines the representation space. The diffusion network learns the reverse stochastic process inside that representation.

15. SAMPLERS

DDPM, DDIM, and faster sampling

The trained denoiser and the numerical sampling procedure are related but not identical components.

Idea	High-level description	Why care?
DDPM sampling	Stochastic reverse transitions, often with many steps.	Canonical formulation and useful teaching baseline.
DDIM	Constructs a non-Markovian/ deterministic-style path for η=0.	Can use fewer steps and enables more deterministic trajectories.
Modern solvers/schedulers	Use improved discretizations or ODE/ SDE-inspired updates.	Reduce neural-network evaluations while preserving quality.

Design lesson

You can often change the **sampler** at inference without retraining the entire denoiser, provided the parameterization and schedule assumptions are compatible.

Do not confuse: number of training timesteps, number of sampling steps, and number of neural-network parameters are three different quantities.

16. USE CASES

Where diffusion models are useful

Diffusion is a general generative mechanism, not only a text-to-image technique.

Use case	Condition / input	Generated or restored output
Text-to-image	Prompt embeddings	Novel image consistent with text.
Inpainting	Image + mask + prompt	Missing/edited region consistent with surroundings.
Super-resolution	Low-resolution image	Higher-resolution plausible reconstruction.
Image-to-image	Source image + condition	Controlled transformation while preserving structure.
Audio / speech generation	Text, spectrogram context, audio features	Waveform or intermediate acoustic representation.
Video	Text/images/frames	Temporally coherent frame sequence.
Scientific generation	Physical/chemical constraints	Candidate structures, molecules, proteins, trajectories, etc.

Why diffusion fits editing. The process naturally supports starting from partially corrupted or constrained representations, making controlled reconstruction tasks especially natural.

17. DESIGN THOUGHT PROCESS

How do I design a diffusion model?

Start from the data geometry and the constraints of the application, not from a fashionable architecture.

1. **Choose the representation.** Raw pixels, spectrogram, latent code, coordinates, sequence embeddings?
2. **Choose the corruption process.** Gaussian diffusion is standard for continuous data; discrete data may need different formulations.
3. **Choose the prediction target.** Noise, x0, v, or score.
4. **Choose a backbone.** MLP for toy vectors, U-Net for images, transformer/DiT for patch or latent sequences.
5. **Condition if needed.** Class label, text embedding, mask, source image, control signal.
6. **Choose schedule and sampling strategy.** Balance quality, stability, and latency.
7. **Define evaluation.** Visual quality alone is not enough; use task-specific metrics and human evaluation where appropriate.

Two-way door choices. Sampler, number of sampling steps, guidance scale, and some conditioning settings are relatively easy to change. Representation and backbone architecture are much more expensive choices because they affect training.

18. PRACTICAL LAB

A tiny diffusion model students can train

We will learn an 8-cluster 2-D distribution. This isolates the diffusion mechanism from expensive image computation.

Training data

12,000 points sampled from eight Gaussian clusters placed on a ring.

Shape: **[N, 2]**

Model

A small MLP predicts two noise values and is conditioned on a 32-dimensional sinusoidal timestep embedding.

Input: **[B, 34]** → Output: **[B, 2]**

Why this is a good teaching model

- No dataset download.
- No GPU required.
- The mathematics is exactly the same forward noising idea used for images.
- Students can visibly compare the learned distribution to the training distribution.
- The exact supplied script trained here in about **7 seconds** on CPU; including sampling and plotting, the full run was about **13 seconds**.

Classroom question. If the input has only 2 dimensions, why can a model still learn a complicated distribution? Because complexity lies in the probability density over the space, not only in the number of dimensions.

19. PYTORCH CODE

Part 1: data and diffusion schedule

The dataset is synthetic; the schedule defines the fixed forward process.

```
import math, time
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

torch.manual_seed(7)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def make_ring_dataset(n=12000, clusters=8, radius=4.0, std=0.28):
    ids = torch.randint(0, clusters, (n,))
    angles = 2 * math.pi * ids.float() / clusters
    centers = torch.stack([
        radius * torch.cos(angles),
        radius * torch.sin(angles)
    ], dim=1)
    return centers + std * torch.randn(n, 2)

x_data = make_ring_dataset().to(device)

T = 100
beta = torch.linspace(1e-4, 0.08, T, device=device)
alpha = 1.0 - beta
alpha_bar = torch.cumprod(alpha, dim=0)
alpha_bar_prev = torch.cat([torch.ones(1, device=device), alpha_bar[:-1]])
posterior_variance = beta * (1.0 - alpha_bar_prev) / (1.0 - alpha_bar)
```

What to notice

- **beta** grows from very small noise to larger noise.
- **alpha = 1 - beta** is per-step signal retention.
- **alpha_bar** is cumulative signal retention.
- **posterior_variance** is used in the stochastic reverse sampler.

x_{data}: [12000, 2] beta: [100] alpha-bar: [100]

20. PYTORCH CODE

Part 2: timestep conditioning and denoiser

One shared network handles every noise level by receiving a representation of t.

```
def timestep_embedding(t, dim=32):
    half = dim // 2
    freq = torch.exp(
        -math.log(10000) * torch.arange(half, device=t.device).float()
        / max(half - 1, 1)
    )
    angles = t.float().unsqueeze(1) * freq.unsqueeze(0)
    return torch.cat([torch.sin(angles), torch.cos(angles)], dim=1)

class NoisePredictor(nn.Module):
    def __init__(self, time_dim=32, hidden=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2 + time_dim, hidden), nn.SiLU(),
            nn.Linear(hidden, hidden), nn.SiLU(),
            nn.Linear(hidden, hidden), nn.SiLU(),
            nn.Linear(hidden, 2),
        )

    def forward(self, x_t, t):
        t_emb = timestep_embedding(t, 32)
        return self.net(torch.cat([x_t, t_emb], dim=1))

model = NoisePredictor().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=2e-3)
```

Tensor shapes for B=256

Tensor	Shape	Meaning
x_t	[256, 2]	Noisy 2-D points.
t	[256]	Integer timestep per point.
t_emb	[256, 32]	Sinusoidal timestep features.
concat	[256, 34]	Point + time information.
output	[256, 2]	Predicted Gaussian noise.

21. PYTORCH CODE

Part 3: the complete training idea

Every iteration manufactures its own supervised example by adding known Gaussian noise.

```
def add_noise(x0, t):
    eps = torch.randn_like(x0)
    a_bar = alpha_bar[t].unsqueeze(1)
    x_t = torch.sqrt(a_bar) * x0 + torch.sqrt(1.0 - a_bar) * eps
    return x_t, eps

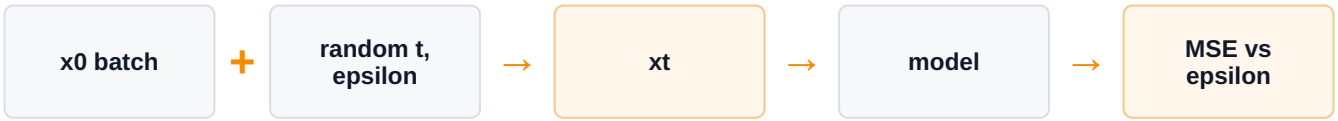
STEPS = 2500
BATCH_SIZE = 256

for step in range(STEPS):
    idx = torch.randint(0, len(x_data), (BATCH_SIZE,), device=device)
    x0 = x_data[idx]
    t = torch.randint(0, T, (BATCH_SIZE,), device=device)

    x_t, true_noise = add_noise(x0, t)
    predicted_noise = model(x_t, t)
    loss = ((predicted_noise - true_noise) ** 2).mean()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if step % 500 == 0:
        print(step, loss.item())
```



The clever part: there are no manually labeled denoising targets. The corruption process automatically creates unlimited training pairs because we know exactly which noise we injected.

22. PYTORCH CODE

Part 4: reverse sampling

Now the model weights are frozen. We repeatedly use the predicted noise to compute a cleaner sample.

```
@torch.no_grad()
def sample(n=3000):
    model.eval()
    x = torch.randn(n, 2, device=device)     # start from x_T

    for i in reversed(range(T)):
        t = torch.full((n,), i, device=device, dtype=torch.long)
        eps_theta = model(x, t)

        a_t = alpha[i]
        a_bar_t = alpha_bar[i]
        b_t = beta[i]

        mean = (1.0 / torch.sqrt(a_t)) * (
            x - (b_t / torch.sqrt(1.0 - a_bar_t)) * eps_theta
        )

        if i > 0:
            z = torch.randn_like(x)
            x = mean + torch.sqrt(posterior_variance[i]) * z
        else:
            x = mean

    return x.cpu()
```

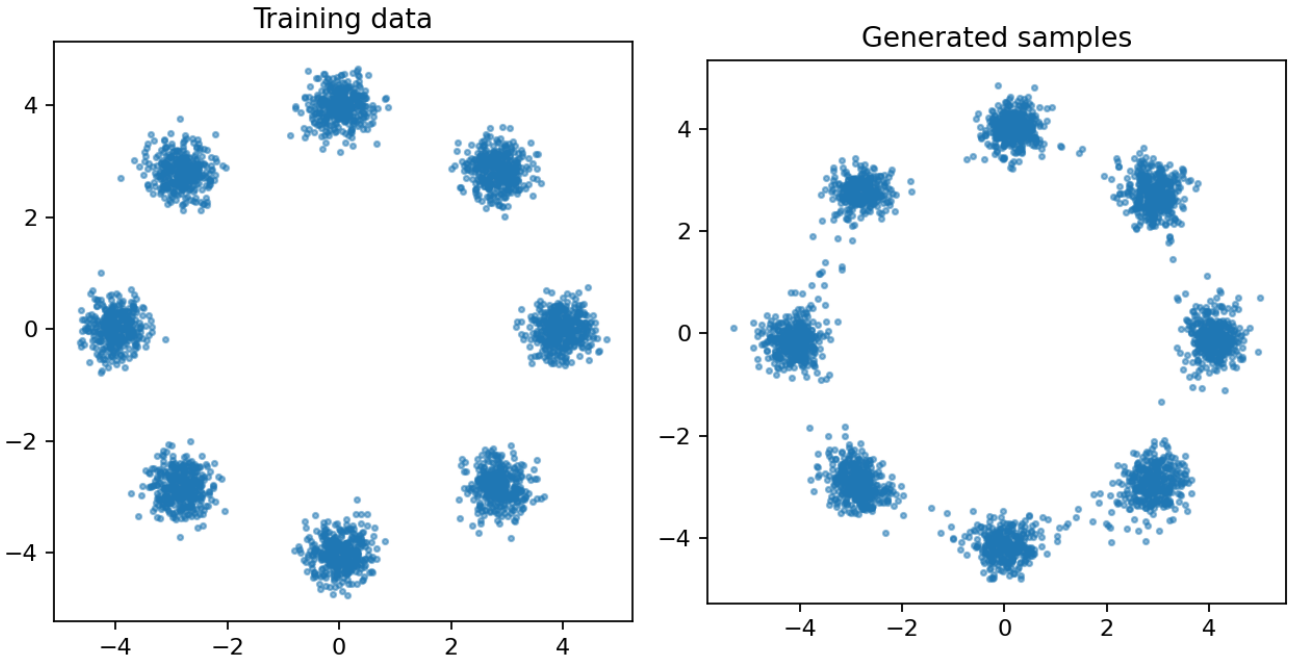
Read the loop conceptually

1. Begin with standard Gaussian noise.
2. Ask the network what noise component it sees at timestep i.
3. Use the DDPM reverse mean formula to remove an estimated amount of that noise.
4. Add the correct amount of stochasticity except at the final step.
5. Repeat until t=0.

23. LAB RESULT

Does the tiny model actually learn?

Yes. The generated points recover the eight-mode geometry of the training distribution.



Result from the exact supplied script in this environment. The generated distribution is not identical point-for-point; it learns the probability structure.

Observed training

2500 optimization steps; CPU training phase approximately 7 seconds in this environment.

What success means

Generated samples occupy the same eight modes with similar spread. Memorizing individual coordinates is unnecessary.

Why MSE need not go to zero. At high noise levels, multiple clean samples can plausibly explain a noisy point. The optimal predictor is statistical; irreducible uncertainty remains.

24. MOVING TO IMAGES

How would we turn this into an image diffusion model?

The training algorithm barely changes. The representation and denoiser become much larger.

Toy lab	Image model
x0 shape [B, 2]	x0 shape [B, C, H, W] or latent [B, C, h, w]
MLP denoiser	U-Net or diffusion transformer
Time embedding concatenated	Time embedding injected into many blocks
No semantic conditioning	Optional text/class/image conditioning
Scatter plot evaluation	Image quality, prompt alignment, diversity, task metrics, human evaluation

The same core training step

```
noise = torch.randn_like(images)
t = random_timesteps(batch)
noisy = sqrt(alpha_bar[t]) * images + sqrt(1-alpha_bar[t]) * noise
pred = unet(noisy, t, condition)
loss = mse(pred, noise)
```

What becomes expensive? The neural network evaluation. High-resolution spatial tensors and many denoising steps dominate compute, which is why latent diffusion and accelerated samplers matter.

25. COMMON MISCONCEPTIONS

What students often get wrong

These distinctions prevent most early confusion.

Misconception	Correction
"The model adds noise during generation."	The model predicts a denoising quantity. The sampler may also inject controlled random noise in intermediate reverse steps.
"There is one neural network for every t."	Usually one shared network is conditioned on t.
"alpha-bar is a trainable weight."	It comes from the chosen noise schedule.
"The model learns to reconstruct each training image."	It learns statistical denoising across the distribution and noise levels.
"Diffusion always works in pixels."	Latent diffusion works in compressed learned representations.
"Text is concatenated to pixels."	Modern text-conditioned models usually encode text and inject it through mechanisms such as cross-attention.
"More sampling steps always means better."	Sampler quality and schedule matter; returns eventually diminish.

26. EXAM / DISCUSSION QUESTIONS

Subjective questions with light mathematics

Use these to test whether students understand the process rather than merely remembering terminology.

Q1. Forward diffusion. A diffusion process has $\bar{\alpha}_t$=0.81. Write the equation used to construct x_t directly from x_0 and Gaussian noise ε. What percentage-like intuition can you give for signal vs noise scale? Why are the coefficients 0.9 and approximately 0.436 rather than 0.81 and 0.19?

Q2. Training. Explain why predicting noise with mean squared error is sufficient to train a generative model even though the dataset contains no "correct generated image" labels. Describe exactly which quantities are sampled, which are known, and which parameters change.

Q3. Architecture design. You want to move from a 2-D toy diffusion model to text-conditioned 256x256 image generation. Explain at least four architectural/system changes you would make and why. Include the roles of a U-Net or DiT, timestep embeddings, text embeddings/cross-attention, and latent diffusion.

Short numerical check

If β_t=0.04, then α_t=0.96. One forward step can be sampled as $x_t = \sqrt{0.96}\, x_{t-1} + \sqrt{0.04}\, \varepsilon$. The noise coefficient is 0.2, not 0.04, because 0.04 is the **variance**.

27. ONE-PAGE REVISION

Diffusion in one page

If students can reconstruct the story below, they understand the essential mechanism.

$$\textbf{Forward: } x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{(1-\bar{\alpha}_t)}\varepsilon$$

$$\textbf{Train: minimize } \| \varepsilon - \varepsilon_\theta(x_t, t, c) \|^2$$

Data → Known corruption q → Noise levels → Learn denoiser

Gaussian noise → Repeated reverse updates → Generated sample

Term	Remember
βt	Per-step noise variance.
αt	1 - βt.
α-bar	Cumulative signal retention.
t embedding	Tells the shared network the noise level.
U-Net / DiT	Denoising backbone for high-dimensional structured data.
Cross-attention	Lets latent/image features use text or other conditioning.
CFG	Strengthens conditional direction during sampling.
Latent diffusion	Runs diffusion in a compressed learned space.

28. REFERENCES

Foundational reading

A short path from the original diffusion formulation to modern latent and transformer-based models.

1. Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., & Ganguli, S. (2015). **Deep Unsupervised Learning using Nonequilibrium Thermodynamics.** ICML.
2. Ho, J., Jain, A., & Abbeel, P. (2020). **Denoising Diffusion Probabilistic Models.** NeurIPS.
3. Song, J., Meng, C., & Ermon, S. (2021). **Denoising Diffusion Implicit Models.** ICLR.
4. Song, Y. et al. (2021). **Score-Based Generative Modeling through Stochastic Differential Equations.** ICLR.
5. Nichol, A. & Dhariwal, P. (2021). **Improved Denoising Diffusion Probabilistic Models.** ICML.
6. Dhariwal, P. & Nichol, A. (2021). **Diffusion Models Beat GANs on Image Synthesis.** NeurIPS.
7. Rombach, R. et al. (2022). **High-Resolution Image Synthesis with Latent Diffusion Models.** CVPR.
8. Peebles, W. & Xie, S. (2023). **Scalable Diffusion Models with Transformers.** ICCV.

Suggested teaching order. Teach the forward closed form and epsilon-prediction loss first. Only after students can explain the toy model should you introduce U-Nets, conditioning, classifier-free guidance, latent diffusion, and faster samplers.

The accompanying Python script is intentionally educational rather than production optimized. Its value is that every line corresponds directly to a mathematical component in these notes.